Technology Stack and Roadmap

Travel Itinerary and Expense Organizer

A MySQL / MySQL Workbench database project for a team of three

32	41	31	4	399
tables	foreign keys	triggers	views	sample rows

Contents: 1 Stack at a glance | 2 Architecture | 3 Why this stack | 4 SQL features used | 5 Application layer and setup | 6 Roadmap and team | 7 What to say about the stack in the viva

Prepared 27 October 2026. Team members: _____, _____, _____

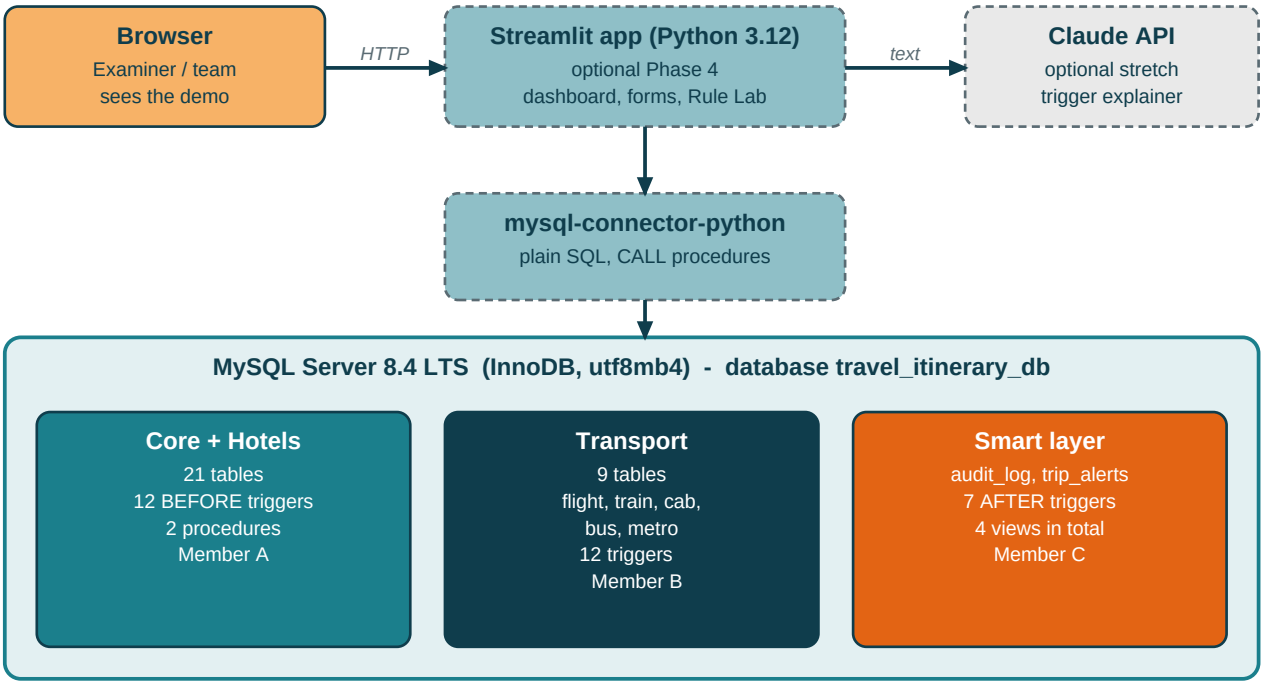
1. Stack at a glance

Required items are everything needed for the Workbench submission. **Optional** items are the stretch layer that makes the demo stronger; the project is complete without them.

Layer	Technology	Status	What it does for us
Database server	**MySQL Community Server 8.4 LTS**	**Required**	Stores all 32 tables, runs the 31 triggers, procedures and views. 8.4 is the supported long-term release (MySQL 8.0 reached end of life in April 2026). Scripts also run on 8.0.16 or newer.
Storage engine	InnoDB	**Required**	Needed for foreign keys, transactions and row locking (SELECT ... FOR UPDATE).
Client and modelling	**MySQL Workbench**	**Required**	Runs the scripts, shows result grids for screenshots and draws the EER diagram (Database, Reverse Engineer).
Language	**SQL, MySQL dialect**	**Required**	DDL, DML, DQL, TCL, plus stored program language for triggers and procedures.
Character set	utf8mb4	**Required**	Full Unicode for names and symbols.
Version control	Git and GitHub	**Required**	Shared repository; one branch per member; pull requests; the README is the project handbook.
Application language	Python 3.12	**Optional**	Thin app on top of the database; all business rules stay in MySQL.
Web interface	Streamlit	**Optional**	Dashboard, timeline, booking form and Rule Lab without writing front-end code.
Database driver	mysql-connector-python	**Optional**	Official driver; plain SQL keeps the queries visible to the examiner.
Data and charts	pandas, Plotly	**Optional**	Query results as tables; budget-versus-spend and transport-mode charts.
Configuration	python-dotenv	**Optional**	Keeps the database password out of the code.
Reproducible setup	Docker Compose, image mysql:8.4	**Optional**	Starts a ready database on any laptop with one command.
AI explainer	Claude API (anthropic package)	**Optional**	Stretch: explains each trigger in plain English from its source code. Needs an API key; written explanations are the fallback.

2. Architecture

The database is the product. Every rule (no double booking, no overlapping activities, carrier must match transport mode, budget alerts) lives inside MySQL, so any tool that writes data is protected by the same rules. Dashed boxes are optional.



Development tools

Data flow in one sentence: a user action in the app becomes plain SQL or a `CALL` to a stored procedure; BEFORE triggers inspect the row and either accept it or raise an error; AFTER triggers then write audit entries and budget alerts; views turn the stored rows into dashboards.

3. Why this stack

Decision principles

Principle	What it means for the choice
The database does the work	Rules are triggers, constraints and procedures, so the grading focus (DBMS design) stays visible and no client can bypass it.
Free and legal	MySQL Community, Workbench, Python, Streamlit and GitHub cost nothing. Only the optional AI explainer needs an API key.
Explainable in a viva	Plain SQL, no ORM, no framework magic. Every query can be shown and discussed line by line.
Low risk for three people	One database, one language for the core, one optional thin app layer. Nothing needs a second team or a server.
Reproducible	Single build file for MySQL, optional Docker, and a README with the run order.

Alternatives considered

Alternative	Why it was not chosen
PostgreSQL	Excellent, but the course tool is MySQL Workbench. Switching adds risk and no extra marks.
SQLite	No stored procedures and no SIGNAL, so the 31 business-rule triggers could not be written the same way.
MongoDB	The data is strongly relational and depends on keys, constraints and joins. A document store removes the main learning goals.
Firebase or Supabase	They hide the schema design work that is being graded.
Django or Flask with an ORM	An ORM hides the SQL. Streamlit with plain SQL is faster to build and easier to explain.
React with Node	A full front end is a large extra workload for three people and adds nothing to the database marks.

4. SQL features used

This is the part of the stack that carries the marks. Everything below is implemented in the scripts and demonstrated in the demo files.

Category	Features used	Where to see it
DDL	CREATE TABLE with PRIMARY KEY, UNIQUE, CHECK, ENUM, DEFAULT; 41 FOREIGN KEYs with CASCADE, RESTRICT and SET NULL; composite keys; indexes; views; DROP and rebuild	Scripts 01, 03, 05
DML	Multi-row INSERT in dependency order, UPDATE, DELETE with cascading effects	Sample data; demos
DQL	Joins and LEFT JOINs, anti-joins with NOT EXISTS, subqueries, GROUP BY with HAVING, UNION ALL, CASE, window functions (SUM OVER), GROUP_CONCAT, TIMESTAMPDIFF	25 named queries
TCL	START TRANSACTION, COMMIT, ROLLBACK, SELECT ... FOR UPDATE (row locking)	sp_book_room, sp_create_trip; demo rule tests
Triggers	24 BEFORE triggers that reject bad rows with SIGNAL SQLSTATE '45000'; 7 AFTER triggers that write audit rows and budget alerts; NEW and OLD; INSERT IGNORE	Triggers section of the README
Stored procedures	Parameters (IN and OUT), DECLARE, WHILE loop, EXIT HANDLER with ROLLBACK and RESIGNAL	sp_create_trip, sp_book_room
Metadata	information_schema.tables, referential_constraints, triggers; SHOW CREATE TRIGGER	Demo 06 quick check and Section D

Modelling techniques that the tools must support

Technique	Implementation
IS-A specialisation	places to hotels; transport_bookings to flight, train, cab, bus and metro detail tables sharing one key
Weak entities	hotel_payments, hotel_reviews, transport_passengers
Many-to-many with attributes	hotel_amenities (is_complimentary), trip_members (role), reviews
Composite foreign key	rooms (room_type_id, hotel_id) to room_types
Derived data in views	nights, balance due, journey duration, budget used

5. Application layer and setup

Optional app packages

Package	Purpose
streamlit	Web UI: dashboard (v_trip_cost_summary), timeline (v_trip_timeline), hotel search, transport, expenses, Rule Lab
mysql-connector-python	Runs the SQL and calls sp_book_room and sp_create_trip. MySQL 8.4 uses caching_sha2_password by default, which this driver supports.
pandas	Turns query results into tables for display
plotly	Charts: budget versus spend, spend by transport mode
python-dotenv	Reads database settings from a .env file kept out of Git
anthropic (optional)	Sends a trigger's source to the Claude API for a plain-English explanation

requirements.txt (pin exact versions with pip freeze once everything works)

```
streamlit
mysql-connector-python
pandas
plotly
python-dotenv
anthropic        # optional
```

.env (never commit this file; add it to .gitignore)

```
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=root
DB_PASSWORD=your_password_here
DB_NAME=travel_itinerary_db
```

Windows setup, step by step

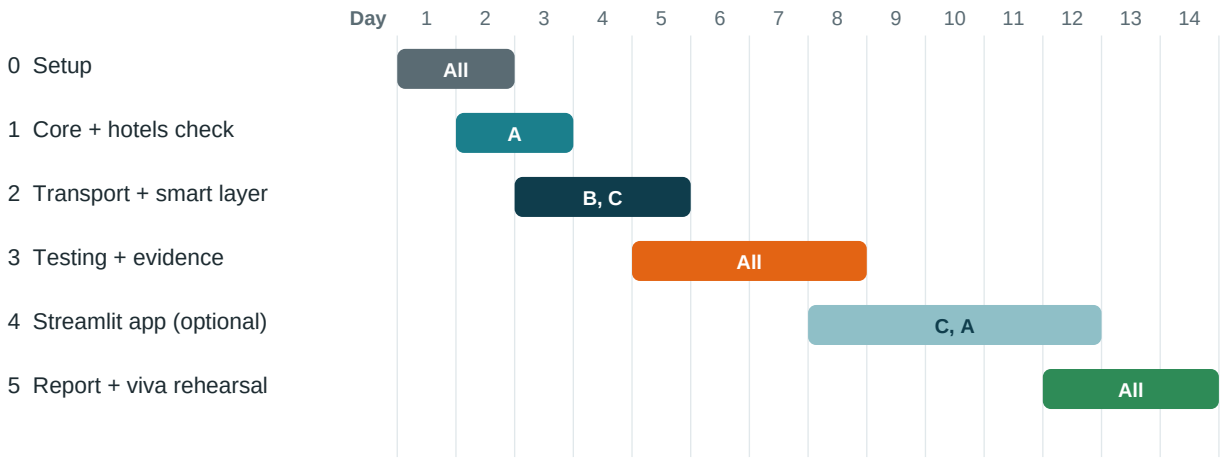
Step	Action
1	Install MySQL Server 8.4 LTS with the MySQL Installer. Keep port 3306, set a root password and tick 'Configure as a Windows service'.
2	Press Win+R, run services.msc and check that the MySQL service shows Running. If not, right-click it and choose Start.
3	Open MySQL Workbench, connect to 127.0.0.1:3306 as root.
4	File, Open SQL Script, choose 00_build_everything.sql and click the lightning bolt. The last grid must show 32 tables, 41 foreign keys, 31 triggers, 4 views.
5	Optional app: python -m venv .venv, then .venv\Scripts\activate, then pip install -r requirements.txt, then streamlit run app/main.py.

Optional Docker Compose (not yet tested; try it only if time allows)

```
services:
  db:
    image: mysql:8.4
    environment:
      MYSQL_ROOT_PASSWORD: your_password_here
    ports: ["3306:3306"]
    volumes:
      - ./sql/00_build_everything.sql:/docker-entrypoint-initdb.d/00_build.sql
```

6. Roadmap and team

A 14-day plan. If the deadline is shorter, merge Phases 3 and 5 and drop Phase 4.



Phase	Goal	Done when
0 Setup (days 1-2)	Everyone has MySQL running, Workbench connected and the GitHub repo cloned	00_build_everything.sql runs on every laptop
1 Core + hotels (2-3)	Compare generated tables with the ER diagram column by column	Member A signs off the comparison
2 Transport + smart layer (3-5)	Read and test every trigger of your module	Quick check: 32, 41, 31, 4
3 Testing + evidence (5-8)	Run all three demo files, take screenshots, export the EER diagram	Every expected result matches
4 App (8-12, optional)	Streamlit dashboard, booking form, Rule Lab, AI trigger explainer	App runs from a clean clone
5 Report + viva (12-14)	Final report, slides, rehearsal on a clean laptop	12-minute demo runs without errors

Who owns what

Member	Module and tools	Main deliverables
A	Core and hotels (21 tables, 12 triggers). Tools: SQL, Workbench, Git owner	Verified schema, EER diagram, amenities and hotel-rule demo
B	Transport and public transport (9 tables, 12 triggers). Tools: SQL, Workbench	Transport model, view v_trip_transport, transport demo
C	Smart layer, queries, report (2 tables, 7 AFTER triggers, 2 views). Tools: SQL, Python and Streamlit (optional)	Alerts and audit demo, dashboards, report, slides, optional app

7. What to say about the stack in the viva

Four sentences every member should be able to say.

Why MySQL?

It is the course database, free, and it supports everything the design needs: foreign keys with referential actions, CHECK constraints, triggers with SIGNAL, stored procedures, views and transactions with row locking. We use the 8.4 LTS release because 8.0 reached end of life in April 2026.

Why is the logic inside the database?

Rules written as triggers and constraints apply to every client. A booking made from Workbench, from the app or from a script is checked in exactly the same way, so the data cannot become inconsistent.

Why plain SQL and no framework?

It keeps every query visible and explainable. The optional Streamlit app only sends SQL and displays results; it adds no business logic.

What is optional and what is not?

MySQL, Workbench and SQL deliver the complete project. Python, Streamlit, Docker and the AI explainer are extras; the demo works without them.

Honest limits to mention

Limit	How we handle it
Scripts were verified by simulation and not yet run on the team's MySQL server	The quick-check counts at the end of the build script are the acceptance test on day one
Demo passwords use SHA2	A real system needs bcrypt or argon2; stated as a limitation
Carrier names are real, times and fares are sample values	Labelled as illustrative sample data
Workbench 26.x may not include the EER editor	Use classic Workbench 8.0.x only to draw the diagram